

BÁO CÁO BÀI TẬP LỚN NHÓM 10

Hệ thống đấu giá trực tuyến BidShift

1. Mục tiêu và phạm vi thực hiện

Mục tiêu: Dự án xây dựng hệ thống đấu giá trực tuyến cho phép seller đăng sản phẩm, bidder tham gia đặt giá theo thời gian thực, admin quản trị dữ liệu và hệ thống tự động kết thúc phiên khi đến hạn. Mục tiêu chính là chứng minh khả năng thiết kế hướng đối tượng, tổ chức kiến trúc nhiều tầng, xử lý đồng thời và triển khai giao diện JavaFX có kết nối server thật.

- **Phạm vi thực hiện (bắt buộc):** đăng ký/đăng nhập, phân quyền Bidder/Seller/Admin, quản lý sản phẩm, tạo phiên đấu giá, đặt giá hợp lệ, kết thúc phiên và xử lý lỗi nghiệp vụ.
- **Phạm vi thực hiện (nâng cao):** xác thực email OTP, quên mật khẩu, auto-bidding, anti-sniping, cập nhật realtime, biểu đồ lịch sử giá, giỏ hàng/checkout, giao hàng, thông báo và chatbot hỗ trợ bằng Gemini API.
- Dữ liệu được lưu trong MySQL; server Java xử lý nghiệp vụ trung tâm; client JavaFX chỉ gửi request, hiển thị dữ liệu và lắng nghe sự kiện realtime.

2. Kiến trúc tổng thể hệ thống

Hệ thống được tổ chức theo mô hình client-server kết hợp MVC/DAO. Phần shared chứa model, DTO và message dùng chung để client và server thống nhất kiểu dữ liệu. Server chịu trách nhiệm kiểm tra nghiệp vụ, transaction, broadcast sự kiện; client tập trung vào giao diện và điều hướng màn hình.

Tầng	Thành phần chính	Vai trò
Client/UI	JavaFX, FXML Controller, SceneNavigator, ServerConnector	Hiển thị giao diện, gửi request socket/REST, nhận event realtime và cập nhật UI bằng Observer.
Shared	model, dto, network, factory, observer, config	Định nghĩa Entity/DTO/Request/Response/AuctionEvent, Factory tạo User/Item và cấu hình AppConfig.
Server core	ServerMain, ClientHandler, ClientManager, AuctionScheduler, AutoBidManager	Nhận kết nối, xử lý action, chạy lịch phiên, auto-bid, broadcast sự kiện cho các client.
DAO/CSDL	UserDAO, ItemDAO, AuctionSessionDAO, BidDAO, AutoBidDAO, CartDAO	Truy cập MySQL bằng JDBC/PreparedStatement; các thao tác quan trọng dùng transaction.
REST phụ trợ	RestApiServer, AuthRestHandler, ChatRestHandler	Cung cấp API OTP/quên mật khẩu/chat AI; session token được quản lý bằng SessionRegistry.

3. Chức năng đạt được, hướng giải quyết và lý do lựa chọn

Chức năng	Hướng giải quyết trong code	Lý do lựa chọn
Quản lý người dùng	User/UserDAO lưu tài khoản, role và trạng thái xác thực; PasswordHasher dùng PBKDF2WithHmacSHA256; AdminController quản lý user.	Tách rõ model/DAO/UI, bảo mật mật khẩu tốt hơn lưu plaintext, dễ mở rộng role.
Sản phẩm & phiên	Item là abstract class, Electronics/Art/Vehicle/OtherItem kế thừa; AuctionSession lưu thời gian, trạng thái, winner, checkoutStatus.	Tận dụng kế thừa/đa hình cho danh mục sản phẩm, phiên đấu giá độc lập với thông tin item.
Đặt giá	BidDAO.placeBidResult() dùng transaction và SELECT ... FOR UPDATE; trả PlaceBidResult với BidFailureReason.	Tránh race condition/lost update khi nhiều bidder đặt giá cùng lúc; client nhận lỗi rõ ràng.
Auto-bidding	AutoBidManager dùng PriorityQueue và khóa AUCTION_LOCKS theo auctionId; gọi BidDAO.placeBidResult() để tái sử dụng luật bid.	Không nhân đôi luật nghiệp vụ, xử lý cạnh tranh tự động theo thứ tự và an toàn đồng thời.
Kết thúc phiên	AuctionScheduler chạy định kỳ, chuyển OPEN/RUNNING/FINISHED/CANCELED/PAID, tạo CartItem cho winner và xử lý quá hạn checkout.	Tách tác vụ nền khỏi UI, đảm bảo phiên tự vận hành dù client không mở màn hình.
Realtime	ClientManager broadcast AuctionEvent qua socket JSON; client đăng ký AuctionEventListener và hiển thị RealtimeToast.	Socket phù hợp kênh kết nối liên tục, giảm polling và phản hồi nhanh khi giá thay đổi.
Hỗ trợ nâng cao	EmailVerificationService/PasswordResetService tạo	Tăng trải nghiệm người dùng, không

Chức năng	Hướng giải quyết trong code	Lý do lựa chọn
	OTP; ChatRestHandler gọi Gemini API qua GEMINI_API_KEY trong AppConfig.	hard-code khóa API, dễ cấu hình khi triển khai.

4. Kiến thức OOP, design pattern và công nghệ áp dụng

- Đóng gói (encapsulation):** các model giữ field private/protected, truy cập qua getter/setter; DAO che giấu chi tiết SQL và Connection.
- Kế thừa và đa hình (inheritance, polymorphism):** User có Admin/Seller/Bidder/RegularUser; Item có Electronics/Art/Vehicle/OtherItem, mỗi lớp có getCategorySpecificInfo().
- Trừu tượng hóa (abstraction):** Entity gom id/createdAt; Item/User là lớp nền cho nhóm đối tượng dùng chung; AuctionEventListener là interface cho cơ chế lắng nghe sự kiện.
- Composition/association:** AuctionSession liên kết Item, Bid, AutoBid, CartItem và Notification thông qua id; server core phối hợp DAO thay vì model tự truy cập DB.
- Design pattern:** MVC ở client JavaFX, DAO cho dữ liệu, **Factory** tạo User/Item/Gson, **Observer** cho realtime event, **Singleton** trong ClientManager/AuctionScheduler/ServerConnector.
- Công nghệ:** Java 17, Maven, JavaFX/FXML, JDBC/MySQL, Gson JSON, TCP Socket, Java HttpServer REST, Jakarta Mail SMTP, Gemini API, JUnit 5 và GitHub Actions.

5. Kiểm thử, triển khai và giới hạn

Các phần dễ lỗi nhất được kiểm thử bằng JUnit: BidDAOTest kiểm tra luật đặt giá, AutoBidManagerTest kiểm tra giá tự động, BidDAOConcurrencyTest kiểm tra tình huống nhiều transaction đặt giá cùng lúc. Workflow Maven chạy clean test để phát hiện lỗi build/test trước khi nộp.

Ứng dụng được đóng gói bằng Maven Shade thành JAR trong thư mục release. Cấu hình host/port, MySQL, SMTP và GEMINI_API_KEY được đọc qua AppConfig từ system property, biến môi trường, .env hoặc application.properties. Điểm còn có thể mở rộng là thêm xác thực mạnh hơn cho toàn bộ socket action, HTTPS khi deploy public và chuẩn hóa thanh toán thật.

Yêu cầu đề bài	Minh chứng trong dự án
OOP	Entity, User/Item abstract class, kế thừa danh mục, interface AuctionEventListener, Factory tạo object.
Networking & MVC	Client JavaFX/FXML theo controller; ServerConnector gửi request; ClientHandler xử lý; DAO tách dữ liệu.
Chức năng nâng cao	Auto-bidding, concurrent bidding, anti-sniping, realtime update, biểu đồ lịch sử giá và Gemini chat.
Tích hợp/triển khai	Maven build, JAR release, MySQL schema, file cấu hình ngoài mã nguồn, GitHub Actions chạy test.

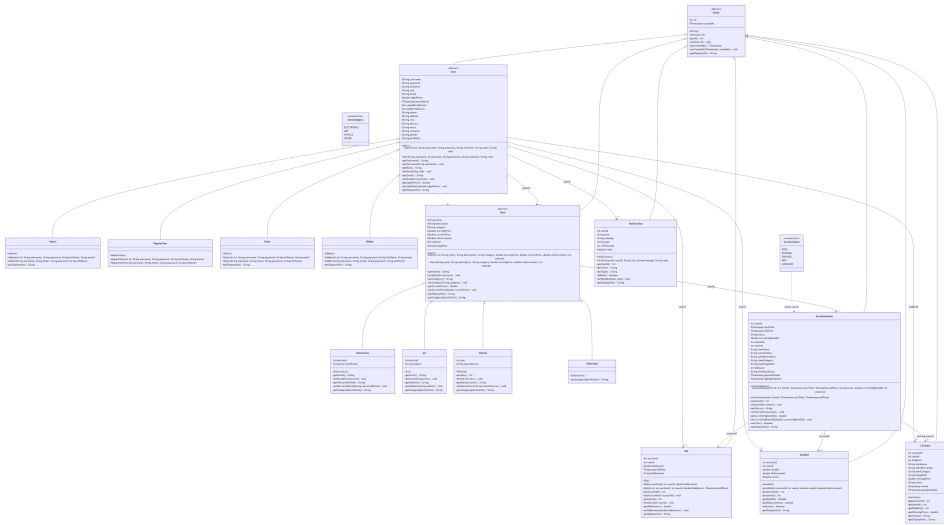
6. Phân chia công việc

Thành viên	Nhiệm vụ chính	Ghi chú
Trần Quốc Hưng	Thiết lập project Maven, networking client-server, socket request/response, database/migration, DAO, xử lý logic backend, đấu giá realtime, auto-bid, scheduler, admin panel, AI chat, email verification, checkout, test, build/release JAR và tài liệu chạy demo.	Trưởng nhóm
Lê Khánh Huyền	Xây dựng module User Management ban đầu, UI/UX Login và Register, refactor luồng đăng nhập/đăng ký, password reset, password hashing, hỗ trợ trạng thái xác thực email trong User/UserDAO.	
Trần Ngọc Hà	Phát triển Dashboard, layout và styling dashboard, light/dark theme, notification model/DAO, SceneNavigator, CartDAO/delivery status, cấu hình runtime AppConfig, password reset service và verification code DAO.	
Nguyễn Bá Kỳ	Phát triển Auction Room UI, cải tiến hiển thị bid history bằng table view, role selection khi đăng ký, bổ sung starting price/minimum increment cho phiên đấu giá, dark/light theme cho Auction Room	

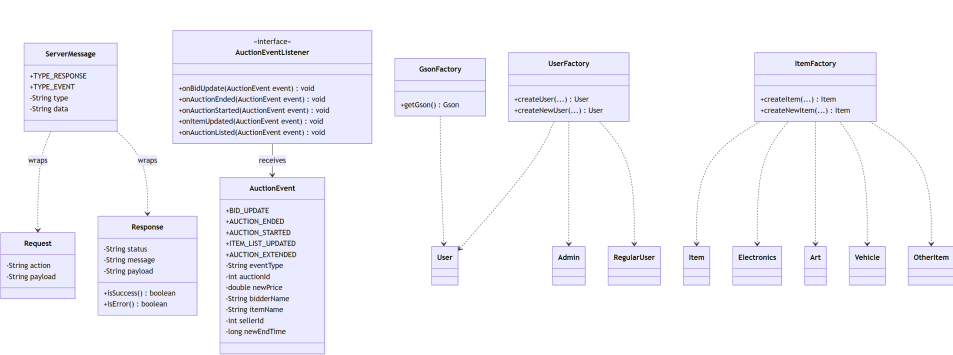
7. Sơ đồ UML chính

Bốn sơ đồ dưới đây tương ứng diagrams số 3, 4, 5 và 6 trong bộ UML: mô hình domain, phần shared/network/factory/observer, lớp phía server và lớp phía client.

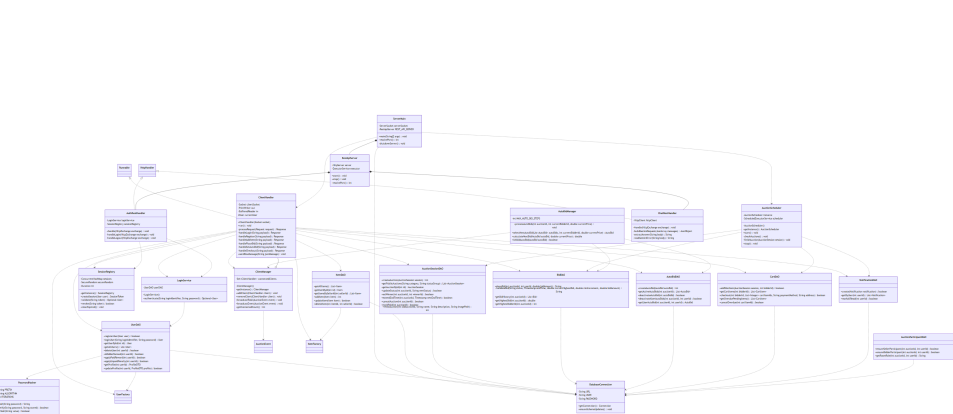
Hình 3. Domain Class Diagram



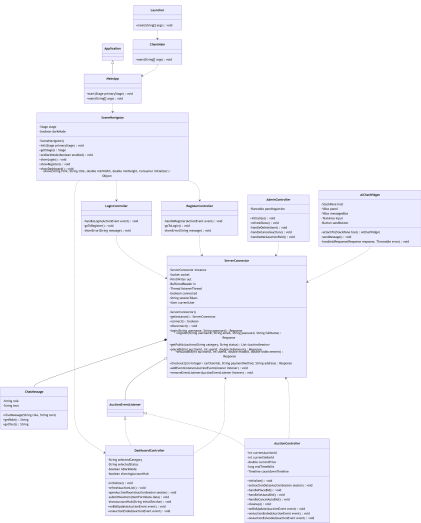
Hình 4. Shared Network, Factory, Observer



Hình 5. Server Class Diagram



Hình 6. Client Class Diagram



8. Kết luận

BidShift đáp ứng các yêu cầu chính của đề bài và có thêm các chức năng nâng cao phục vụ trải nghiệm đấu giá thực tế. Kiến trúc phân tầng giúp nhóm dễ giải thích từng phần khi vấn đề: client gửi request, server xử lý nghiệp vụ, DAO đảm bảo dữ liệu, scheduler/observer duy trì tính tự động và realtime.